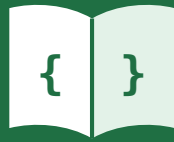

Modül 14 — Spring Boot'ta Test

Ders Notları

Spring Boot Edu



2026-09-24

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Test Piramidi	2
2.2 Bir Slice Neyi Yükler	3
2.3 Test Dublörleri	3
3. Adım Adım Örnekler	4
3.1 Unit Testler: Sade Java	5
3.2 Mockito ile Unit Testler	6
3.3 Web Slice: @WebMvcTest ve MockMvcTester	7
3.4 Veri Slice'ı: PostgreSQL'e Karşı @DataJpaTest	8
3.5 JSON Slice'ı: @JsonTest	8
3.6 Entegrasyon Testleri: @SpringBootTest	9
3.7 Testcontainers	11
3.8 ArchUnit ile Mimari Testleri	12
3.9 Builder'larla Test Verisi	12
4. Sık Yapılan Hatalar ve En İyi Pratikler	13
5. Özet	14
6. İleri Okuma	14

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Test piramidini açıklamak ve bir kod parçası için doğru test seviyesini seçmek
- JUnit 6, AssertJ ve Mockito ile hızlı unit testler yazmak
- Bir katmanı gerçek Spring yapılandırmasıyla test etmek için slice testleri (@WebMvcTest, @DataJpaTest, @JsonTest) kullanmak
- Bean'leri @MockitoBean ve @MockitoSpyBean ile değiştirmek veya gözlemlemek
- @SpringBootTest, RestTestClient ve Testcontainers ile entegrasyon testleri yazmak
- Test verisini builder'larla okunabilir tutmak
- Mimariyi ArchUnit kurallarıyla korumak

Ön koşullar: Modül 06 (JPA) · **Tahmini süre:** 5 saat · Veritabanı testleri için **Docker gerekir**

i Not

Bu modülde ders, **test paketinin kendisidir**. Uygulama (bir sipariş servisi) yalnızca test edilen nesnedir. Her şeyi `./mvnw -pl modules/14-testing/lesson verify` ile çalıştırın, sonra testleri bu notlarla yan yana okuyun.

2. Kavramlar

2.1 Test Piramidi

↪	▲ entegrasyon	(@SpringBootTest + gerçek DB)	az, yavaş, parçaların
	uyduğunu kanıtlar		
	▲▲▲ slice'lar	(@WebMvcTest, @DataJpaTest, ...)	birkaç, her biri tek katman
	▲▲▲▲ unit	(JUnit, Mockito, Spring yok)	çok, milisaniyeler

Bir test ne kadar aşağıdaysa o kadar hızlı ve kesindir. Ne kadar yukarıdaysa gerçek sistem hakkında o kadar çok şey kanıtlar. Sağlıklı bir pakette çok sayıda unit test, birkaç slice test ve az sayıda entegrasyon testi vardır. Bu modülün test paketinin süreleri (yazarın bilgisayarıyla bir çalıştırma):

Test	Seviye	Süre
PriceCalculatorTest (8 durum)	unit	0,06 s
OrderServiceTest	Mockito ile unit	1,1 s

Test	Seviye	Süre
OrderControllerTest	@WebMvcTest slice	0,9 s
OrderConfirmationJsonTest	@JsonTest slice	1,4 s
BookRepositoryTest	@DataJpaTest + PostgreSQL	5,5 s (container başlatma dahil)
OrderFlowIT	@SpringBootTest, gerçek port ve veritabanı	7,7 s

2.2 Bir Slice Neyi Yükler

Bir slice testi, uygulama context'inin bir **parçasını** başlatır: yalnızca bir katmana ait bean'leri.

Annotation	Yükler	Yüklemez
@WebMvcTest(X.class)	X controller'ı, MVC yapılandırması, Jackson, validation, MockMvcTester	servisler, repository'ler, veritabanı
@DataJpaTest	entity'ler, repository'ler, EntityManager, Flyway, TestEntityManager	controller'lar, servisler
@JsonTest	uygulamanın yapılandırmasıyla Jackson, JacksonTester	geri kalan her şey
@SpringBootTest	her şey	—

Eksik işbirlikçiler @MockitoBean ile sağlanır.

2.3 Test Dublörleri

Dublör	Nedir	Bu modülde
Mock	çağrılar kaydeder, programladığınızı döndürür	@Mock, @MockitoBean PaymentClient
Spy	gerçek nesne, çağrılar ayrıca kaydedilir	@MockitoSpyBean AuditLog

Dublör	Nedir	Bu modülde
Fake	basit, çalışan bir gerçekleştirme	bellek içi bir depo (modül 12, 13)

Yalnızca sahibi olmadığınız veya çalıştıramadığınız şeyleri (bir ödeme sağlayıcısı) mock'layın. Ucuz olduğunda gerçeğini kullanın (PriceCalculator, bir Testcontainers veritabanı).

3. Adım Adım Örnekler

Test edilen nesne bir sipariş servisi: Bir kitabı bulur, stoğu kontrol eder, fiyatı hesaplar, ödeme servisinden tahsil eder ve bir denetim kaydı yazar.

```
@Service
public class OrderService {

    private final BookRepository books;
    private final PriceCalculator prices;
    private final PaymentClient payments;
    private final AuditLog audit;

    public OrderService(BookRepository books, PriceCalculator prices, PaymentClient
        payments, AuditLog audit) {
        this.books = books;
        this.prices = prices;
        this.payments = payments;
        this.audit = audit;
    }

    @Transactional
    public OrderConfirmation placeOrder(String isbn, int quantity) {
        Book book = books.findByIsbn(isbn).orElseThrow(() -> new
            BookNotFoundException(isbn));
        book.removeFromStock(quantity); // may throw
        OutOfStockException
        BigDecimal total = prices.total(book.getPrice(), quantity);
        PaymentReceipt receipt = payments.charge(new PaymentRequest("order-" + isbn,
            total));
        audit.record("ordered " + quantity + " x " + isbn + " for " + total);
        return new OrderConfirmation(isbn, book.getTitle(), quantity, total,
            receipt.id());
    }
}
```

3.1 Unit Testler: Sade Java

Fiyat kuralları hiçbir Spring bağımlılığı olmayan bir sınıfta durur:

```
public class PriceCalculator {

    public BigDecimal total(BigDecimal unitPrice, int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("quantity must be positive, was " +
                ↪ quantity);
        }
        BigDecimal gross = unitPrice.multiply(BigDecimal.valueOf(quantity));
        BigDecimal discount = quantity >= 10 ? new BigDecimal("0.10") // 10 %
            ↪ from 10 copies
                : quantity >= 5 ? new BigDecimal("0.05") // 5 %
            ↪ from 5 copies
                : BigDecimal.ZERO;
        return gross.subtract(gross.multiply(discount)).setScale(2,
            ↪ RoundingMode.HALF_UP);
    }
}
```

Testi new'den başka bir şeye ihtiyaç duymaz. Parametrelili bir test, sınırlar dahil her kuralı kapsar:

```
class PriceCalculatorTest {

    private final PriceCalculator calculator = new PriceCalculator();

    @ParameterizedTest(name = "{1} × {0} = {2}")
    @CsvSource({
        "10.00, 1, 10.00", // no discount
        "10.00, 4, 40.00",
        "10.00, 5, 47.50", // 5 % from 5 copies
        "10.00, 10, 90.00", // 10 % from 10 copies
        "89.90, 3, 269.70",
        "9.99, 5, 47.45"}) // 49.95 - 2.4975 = 47.4525 → rounded half up
    void appliesQuantityDiscounts(String unitPrice, int quantity, String expected) {
        assertThat(calculator.total(new BigDecimal(unitPrice),
            ↪ quantity)).isEqualToByComparingTo(expected);
    }

    @ParameterizedTest
    @ValueSource(ints = {0, -1})
    void rejectsQuantitiesBelowOne(int quantity) {
        assertThatIllegalArgument().isThrownBy(() ->
            ↪ calculator.total(BigDecimal.TEN, quantity));
    }
}
```

```
}  
}
```

3.2 Mockito ile Unit Testler

OrderService'in işbirlikçileri vardır. Bir unit testte bunlar **mock**'tur, yine hiçbir Spring context'i olmadan:

```
@ExtendWith(MockitoExtension.class)  
class OrderServiceTest {  
  
    @Mock  
    BookRepository books;  
  
    @Mock  
    PaymentClient payments;  
  
    @Mock  
    AuditLog audit;  
  
    @Captor  
    ArgumentCaptor<PaymentRequest> paymentRequest;  
  
    private OrderService service() {  
        return new OrderService(books, new PriceCalculator(), payments, audit); // the  
        ↪ real calculator  
    }  
  
    @Test  
    void chargesTheDiscountedTotalAndLowersTheStock() {  
        Book book = aBook().isbn("9780134685991").price("10.00").stock(8).build();  
        given(books.findByIsbn("9780134685991")).willReturn(Optional.of(book));  
        given(payments.charge(any())).willReturn(new PaymentReceipt("pay-1", "PAID"));  
  
        OrderConfirmation confirmation = service().placeOrder("9780134685991", 5);  
  
        then(payments).should().charge(paymentRequest.capture());  
        assertThat(paymentRequest.getValue().amount()).isEqualToByComparingTo("47.50");  
        assertThat(confirmation.paymentId()).isEqualTo("pay-1");  
        assertThat(book.getStock()).isEqualTo(3);  
    }  
}
```

- `given(...).willReturn(...)` bir mock'u programlar, `then(mock).should()` bir çağrışı doğrular (BDD stili).
- Bir `ArgumentCaptor` bir çağrının argümanını yakalar, burada ödeme servisine gönderilen tutarı.

- PriceCalculator gerçektir: Ucuz ve deterministiktir, onu mock'lamak için bir neden yoktur.
- Bunların hepsini constructor injection mümkün kılar: Test, constructor'ı kendisi çağırır.

3.3 Web Slice: @WebMvcTest ve MockMvcTester

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    MockMvcTester mvc;

    @MockitoBean // replaces the bean in the Spring
    ↪ context with a mock
    OrderService orders;

    @Test
    void aValidOrderIsCreated() {
        given(orders.placeOrder("9780134685991", 2)).willReturn(new OrderConfirmation(
            "9780134685991", "Effective Java", 2, new BigDecimal("179.80"), "pay-7"));

        assertThat(post("""
            {"isbn": "9780134685991", "quantity": 2}"""))
            .hasStatus(201)
            .hasHeader("Location", "/api/orders/pay-7")
            .bodyJson().extractingPath("$.total").isEqualTo("179.80");
    }

    @Test
    void anInvalidBodyIs400AndNeverReachesTheService() {
        assertThat(post("""
            {"isbn": "", "quantity": 0}"""))
            .hasStatus(400);
        then(orders).should(never()).placeOrder(anyString(), anyInt());
    }
}
```

- Yalnızca OrderController ve MVC altyapısı başlar. Servis bir @MockitoBean'dir.
- MockMvcTester (Spring Framework 6.2+) sunucu olmadan istek gönderir ve AssertJ doğrulamaları sunar: durum, header'lar, JSON yolları.
- Test, yalnızca web katmanının yanlış yapabileceği şeyleri kontrol eder: doğrulama (400), durum kodları, header'lar, hata gövdeleri.

3.4 Veri Slice'ı: PostgreSQL'e Karşı @DataJpaTest

```
@DataJpaTest
@Import(TestcontainersConfiguration.class)
class BookRepositoryTest {

    @Autowired
    BookRepository books;

    @Autowired
    EntityManager entities; // writes test data directly, bypassing
    ↪ the repository

    @Test
    void findsABookByIsbn() {
        entities.persistAndFlush(aBook().isbn("9789999999991").build());

        assertThat(books.findByIsbn("9789999999991")).isPresent();
    }

    @Test
    void listsTheBooksRunningLowWithTheLowestStockFirst() {
        entities.persist(aBook().isbn("9789999999992").stock(1).build());
        entities.persist(aBook().isbn("9789999999993").soldOut().build());
        entities.persistAndFlush(aBook().isbn("9789999999994").stock(50).build());

        assertThat(books.runningLow(2)).extracting(Book::getIsbn)
            .containsSequence("9789999999993", "9789999999992") // and the seeded
            ↪ sold-out Java Puzzlers
            .doesNotContain("9789999999994");
    }
}
```

- @DataJpaTest her testi sonunda **geri alınan** (rollback) bir transaction içinde çalıştırır. Testler birbirinin verisini görmez.
- EntityManager test verisini doğrudan yazar. Böylece test yalnızca test edilen sorguyu kontrol eder.
- Veritabanı H2 değil, gerçek bir PostgreSQL'dir (Testcontainers): order by'lı bir JPQL sorgusu, bir CHECK kısıtı veya PostgreSQL'e özgü bir tip tam olarak üretimdeki gibi davranır.

3.5 JSON Slice'ı: @JsonTest

Para JSON'da kayan noktalı bir sayıya dönüşmemelidir. Record onu string olarak yazar:

```
public record OrderConfirmation(String isbn, String title, int quantity,
    @JsonFormat(shape = JsonFormat.Shape.STRING) BigDecimal
    ↪ total,
    String paymentId) {
}
```

```
@JsonTest
class OrderConfirmationJsonTest {

    @Autowired
    JacksonTester<OrderConfirmation> json;

    @Test
    void writesMoneyAsString() throws Exception {
        var confirmation = new OrderConfirmation("9780134685991", "Effective Java", 2,
            new BigDecimal("179.80"), "pay-7");

        assertThat(json.write(confirmation))
            .extractingJsonPathStringValue("$.total").isEqualTo("179.80");    // not
            ↪ 179.8
    }

    @Test
    void readsTheSameFormatBack() throws Exception {
        var read = json.parseObject("""
            {"isbn":"9780134685991","title":"Effective
            ↪ Java","quantity":2,"total":"179.80","paymentId":"pay-7"}""");

        assertThat(read.total()).isEqualByComparingTo("179.80");
    }
}
```

@JsonTest, uygulamayla aynı Jackson yapılandırmasını kullanır. Değişen bir JsonMapper ayarı burada ortaya çıkar.

3.6 Entegrasyon Testleri: @SpringBootTest

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@Import(TestcontainersConfiguration.class)
class OrderFlowIT {

    @Autowired
    RestTestClient http;
```

```
@Autowired
BookRepository books;

@MockitoBean // the external system: never called
↳ for real in tests
PaymentClient payments;

@MockitoSpyBean // the real bean, but its calls can be
↳ verified
AuditLog audit;

@Test
void anOrderIsPaidStoredAndAudited() {
    given(payments.charge(any())).willReturn(new PaymentReceipt("pay-42", "PAID"));
    int stockBefore = stockOf("9780134685991");

    http.post().uri("/api/orders").contentType(MediaType.APPLICATION_JSON)
        .body("""
            {"isbn": "9780134685991", "quantity": 2}""")
        .exchange()
        .expectStatus().isCreated()
        .expectBody().jsonPath("$.paymentId").isEqualTo("pay-42");

    assertThat(stockOf("9780134685991")).isEqualTo(stockBefore - 2);
    then(audit).should().record("ordered 2 × 9780134685991 for 179.80");
}

@Test
void aFailedPaymentRollsTheStockBack() {
    given(payments.charge(any())).willThrow(new RestClientException("payment service
    ↳ down"));
    int stockBefore = stockOf("9781617297571");

    http.post().uri("/api/orders").contentType(MediaType.APPLICATION_JSON)
        .body("""
            {"isbn": "9781617297571", "quantity": 1}""")
        .exchange()
        .expectStatus().is5xxServerError();

    assertThat(stockOf("9781617297571")).isEqualTo(stockBefore); // @Transactional
    ↳ rolled back
}
```

- RANDOM_PORT gerçek sunucuyu başlatır. RestTestClient (Spring Framework 7) gerçek HTTP istekleri gönderir.
- @MockitoBean PaymentClient tek harici sistemi değiştirir. PostgreSQL ve transaction

dahil geri kalan her şey gerçektir.

- `@MockitoSpyBean` `AuditLog` gerçek bean'i tutar ama testin çağrılarını doğrulamasına izin verir.
- İkinci test, hiçbir unit testin kanıtlamayacağı bir şeyi kanıtlar: Ödeme başarısız olduğunda `@Transactional` stok değişikliğini geri alır.

★ İpucu

Her farklı `@MockitoBean` ve property kombinasyonu yeni bir uygulama context'i oluşturur. Spring önbellekteki tek bir context'i yeniden kullanabilsin diye bunları test sınıfları arasında aynı tutun.

3.7 Testcontainers

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
    ↪ pg18").asCompatibleSubstituteFor("postgres"))
        ↪ .withReuse(true);          // only if ~/.testcontainers.properties says
        ↪ testcontainers.reuse.enable=true

    @Bean
    @ServiceConnection          // Boot derives the DataSource (and Flyway) settings
    ↪ from the container
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }
}
```

- `static final`: JVM başına bir container, önbellekteki tüm test context'leri tarafından paylaşılır.
- `@ServiceConnection`: Boot URL'yi, kullanıcıyı ve parolayı container'dan okur. `@DynamicPropertySource` gerekmez.
- `withReuse(true)`, container'ı test çalıştırmaları **arasında** açık tutar. Ancak bunu yalnızca kendi bilgisayarınızda `~/.testcontainers.properties` içinde `testcontainers.reuse.enable=true` ile açarsanız. CI'da etkisi yoktur. Bu, yerel test çalıştırmalarının saniyeler içinde başlamasını sağlar.

3.8 ArchUnit ile Mimari Testleri

```
@AnalyzeClasses(packages = "com.springbootedu.testing", importOptions =
    ↳ ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule controllersDoNotTalkToRepositories = noClasses()
        .that().areAnnotatedWith(RestController.class)
        .should().dependOnClassesThat().haveSimpleNameEndingWith("Repository")
        .because("controllers go through a service, which owns the transaction");

    @ArchTest
    static final ArchRule pricingIsPlainJava = noClasses()
        .that().resideInAPackage("..pricing..")
        .should().dependOnClassesThat().resideInAPackage("org.springframework..")
        .because("business rules must be testable without Spring");

    @ArchTest
    static final ArchRule noFieldInjection = fields()
        .should().notBeAnnotatedWith(Autowired.class)
        .because("constructor injection makes dependencies explicit and classes
            ↳ testable with new");
}
```

ArchUnit derlenmiş sınıfları okur ve kuralları diğer testler gibi kontrol eder. Bir controller bir repository'yi doğrudan kullanmaya başladığında build şöyle bir mesajla başarısız olur:

```
Architecture Violation [Priority: MEDIUM] - Rule 'no classes that are annotated with
    ↳ @RestController should depend on
classes that have simple name ending with 'Repository', because controllers go through a
    ↳ service, which owns the transaction'
```

3.9 Builder'larla Test Verisi

```
public final class TestBooks {

    private String isbn = "9780000000000";
    private String title = "A Test Book";
    private BigDecimal price = new BigDecimal("10.00");
    private int stock = 5;

    private TestBooks() {
```

```
}

public static TestBooks aBook() {
    return new TestBooks();
}

public TestBooks isbn(String isbn) {
    this.isbn = isbn;
    return this;
}

public TestBooks price(String price) {
    this.price = new BigDecimal(price);
    return this;
}

public TestBooks stock(int stock) {
    this.stock = stock;
    return this;
}

public TestBooks soldOut() {
    return stock(0);
}

public Book build() {
    return new Book(isbn, title, price, stock);
}
}
```

`aBook().stock(0).build()` test için tam olarak neyin önemli olduğunu söyler. Geri kalan her şeyin makul bir varsayılana vardır. `Book`'a yeni bir alan eklendiğinde her test değil, yalnızca builder değişir.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Bir testin asla bugünün tarihine, bir `HashSet`'in sırasına, sabit bir `Thread.sleep`'e veya testlerin çalışma sırasına bağlı olmasına izin vermeyin. Böyle testler **flaky**'dir (kararsızdır): Kod değişmeden zaman zaman başarısız olurlar ve kısa sürede kimse test paketine güvenmez. Bir `Clock` enjekte edin, sırasız doğrulayın, `Awaitility` ile bekleyin ve her testin kendi verisini hazırlamasını sağlayın.

- **Yapın:** Davranışı private ayrıntılar üzerinden değil, public metotlar üzerinden test edin.
- **Yapmayın:** Her şey için `@SpringBootTest` kullanmayın. En yavaş seviyedir.
- **Yapın:** Repository testleri için gerçek bir veritabanı (Testcontainers) kullanın. H2, PostgreSQL'den farklı davranır.
- **Yapmayın:** Sahibi olduğunuz ve ucuza çalıştırabildiğiniz şeyleri mock'lamayın. Mock'lanmış bir `PriceCalculator` hiçbir şey test etmezdi.
- **Yapın:** Testlere bir gereksinim gibi okunan adlar verin: `aFailedPaymentRollsTheS-tockBack`.
- **Yapmayın:** Testin konusundan fazlasını doğrulamayın. Aşırı belirtilmiş testler her zarsız değişiklikte kırılır.

5. Özet

- Test piramidi: çok sayıda hızlı unit test, birkaç slice test, az sayıda entegrasyon testi.
- Unit testler Spring istemez. Mockito işbirlikçileri değiştirir, `ArgumentCaptor` neyin gönderildiğini kontrol eder.
- `@WebMvcTest`, `@DataJpaTest` ve `@JsonTest` her biri bir katmanı gerçek yapılandırmasıyla yükler.
- `RestTestClient`'lı `@SpringBootTest` tüm uygulamayı test eder. `@MockitoBean` harici sistemleri değiştirir, `@MockitoSpyBean` gerçek bean'leri gözlemler.
- `@ServiceConnection`'lı Testcontainers her teste gerçek bir veritabanı verir, `withReuse` onu yerelde hızlandırır.
- Builder'lar test verisini kısa tutar, ArchUnit mimari kararları testlere dönüştürür.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — Testing](#) · [Testing Spring Boot Applications](#)
- [Spring Framework — MockMvcTester](#) · [RestTestClient](#) · [Bean Overriding](#)
- [JUnit User Guide](#) · [Mockito](#) · [AssertJ](#)
- [Testcontainers for Java](#) · [Reusable Containers](#)
- [ArchUnit User Guide](#)
- [Martin Fowler — The Practical Test Pyramid](#)